

# SplitUp Project Review

Expense Manager - technical review PDF

Generated from the local workspace at  
C:/Users/dhaan/OneDrive/Desktop/Expense\_manager

|                |                                                                                      |
|----------------|--------------------------------------------------------------------------------------|
| Review date    | 22 May 2026, 16:07                                                                   |
| Project type   | Full-stack group expense management app with real-time chat                          |
| Backend        | Spring Boot 3.1.0, Java 17, Spring Security, JPA, PostgreSQL driver, WebSocket/STOMP |
| Frontend       | Expo SDK 54, React Native 0.81.5, React 19.1, TypeScript 5.9, Axios, SockJS/STOMP    |
| Code footprint | 79 Java files, 40 TypeScript/TSX files in app/src areas                              |

This review focuses on architecture, current implementation health, verification results, and practical next steps. It does not expose local secrets from application.properties.

## Executive Summary

SplitUp is a capable mobile-first expense sharing product: it combines authentication, group management, invitations, expense transactions, settlement calculation, and chat/presence in one coherent app. The backend has a clear module split, and the frontend is already organized around Expo Router tabs and API service modules.

- Strongest areas: feature coverage, REST/WebSocket split, reusable API modules, password hashing, JWT-based request authentication, and a simple settlement engine.
- Current readiness: backend context test passes; frontend lint currently fails on 4 JSX escaping errors and reports 58 warnings.
- Main risks: low automated test depth, configuration drift between docs/examples and actual PostgreSQL setup, verbose debug logging, and WebSocket security bypass warnings.
- Best next milestone: clean lint errors, add targeted service tests, and formalize environment configuration before expanding features.

## Project Shape

| Area           | What exists                                                                                   | Review note                                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Authentication | JWT signin/signup, BCrypt password encoder, role entities and repositories                    | Good baseline; signup can accept role names from request, so role assignment policy should be reviewed before production. |
| Groups         | Group, member, invite, detail DTOs, controllers, services, repositories                       | Core collaboration model is present and reasonably modular.                                                               |
| Expenses       | Expense and transaction CRUD plus DTOs and frontend screens                                   | Feature surface is broad; money handling should avoid floating-point doubles for production finance logic.                |
| Settlements    | Greedy settlement engine produces debtor-to-creditor balances                                 | Algorithm is simple and useful; add unit tests for rounding, zero balances, and multi-party cases.                        |
| Chat           | STOMP/SockJS WebSocket setup, message service, presence and typing indicators                 | Real-time layer is ambitious; security and lifecycle behavior deserve focused testing.                                    |
| Frontend       | Expo Router tabs for groups, settlements, messages, invitations; auth screens and API modules | App structure is understandable; cleanup pass needed for lint, hooks, duplicate imports, and debug logs.                  |

## Verification Results

| Check             | Result  | Notes                                                                                                        |
|-------------------|---------|--------------------------------------------------------------------------------------------------------------|
| Backend tests     | PASS    | Ran <code>./mvnw.cmd test</code> . Result: 1 test, 0 failures, build success.                                |
| Frontend lint     | FAIL    | Ran <code>npm.cmd run lint</code> . Result: 4 errors and 58 warnings.                                        |
| PDF dependencies  | PASS    | ReportLab and pypdf are available through the bundled Codex Python runtime.                                  |
| Visual PDF render | LIMITED | Poppler/pdftoppm was not installed. Text extraction and structural generation checks were performed instead. |

### Notable Test Output

- Backend booted with the dev profile and connected to PostgreSQL during the context test.
- Spring Security warned that ignoring `/ws` and `/ws/**` is not recommended; `permitAll` inside `HttpSecurity` is preferred.
- The test run executed database migration logic against a configured database, which is risky for a plain `contextLoads` test.
- Frontend lint errors are unescaped apostrophe/quote JSX text in invitations, group detail, and expense detail screens.

## Review Findings

| Severity | Finding                                                   | Evidence                                                                                       | Recommendation                                                                                                 |
|----------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| High     | Frontend lint currently fails.                            | 4 react/no-unescaped-entities errors; 58 warnings from Expo lint.                              | Fix JSX escaping errors first, then clean warnings that affect hooks, imports, and unused values.              |
| High     | Tests can touch a real database.                          | mvnw test started Hikari PostgreSQL and executed production database migration logs.           | Use a test profile with isolated DB/container or H2-compatible setup; keep migrations explicit and reversible. |
| Medium   | WebSocket endpoint bypasses normal security filter chain. | Spring Security warns against web.ignoring() for /ws and /ws/**.                               | Keep /ws permitAll if needed, but avoid web.ignoring(); authenticate STOMP CONNECT headers consistently.       |
| Medium   | Config/documentation drift.                               | README emphasizes PostgreSQL, while application.properties.example uses MySQL dialect and URL. | Align examples, setup guides, and dependency list around the intended database.                                |
| Medium   | Money values use floating point.                          | SettlementEngine and expense transactions use Double/double amounts.                           | Use BigDecimal or integer minor units for financial calculations and persistence.                              |
| Medium   | Excessive debug logging in frontend.                      | Many console.log calls in chat, WebSocket, auth, and group screens.                            | Gate debug logs by environment or replace with a structured logger with production suppression.                |
| Low      | Automated coverage is shallow.                            | Only one backend contextLoads test found; no frontend test script in package.json.             | Add service/controller tests for auth, groups, settlements, invites, and chat message flows.                   |

## Recommended Next Steps

| Priority | Action                                                                                                                     | Expected outcome                                                 |
|----------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 1        | Fix the 4 lint errors and run npm.cmd run lint until errors are zero.                                                      | Frontend is ready for cleaner CI checks.                         |
| 2        | Move backend tests onto a dedicated test profile and database.                                                             | Tests stop mutating or depending on development/production data. |
| 3        | Replace settlement amount doubles with BigDecimal or cents-based integers.                                                 | More reliable money math and fewer rounding surprises.           |
| 4        | Remove Spring Security web.ignoring for WebSocket endpoints and validate STOMP auth flow.                                  | Cleaner security posture and fewer framework warnings.           |
| 5        | Synchronize README, backend README, application.properties.example, and deployment docs.                                   | New setup becomes predictable for another developer.             |
| 6        | Add targeted unit/integration tests around settlement, invite acceptance, group membership, and chat read/unread behavior. | Regression protection around the app's core value.               |

## Closing Assessment

The project is beyond a prototype in feature scope and has a solid full-stack direction. The next quality leap is less about adding screens and more about tightening operational confidence: deterministic tests, consistent environment docs, lint-clean frontend code, and production-safe handling of money and WebSocket authentication.

Overall review: promising and demonstrable, with several practical hardening tasks before treating it as production-ready.